



Implementation Plan: Phone Verification & Nigerian Address System

Overview

Objective: Implement comprehensive phone number verification and structured Nigerian address management for users, organizations, and branches.

Timeline: 2 weeks

Priority: High (critical for Nigerian market)

Dependencies: Authentication, Organization/Workspace management, Branch management

Phase 1: Nigerian Location Database (Days 1-2)

User Story 1.1: Location Data Model

As a system architect, I need a database of Nigerian states and LGAs so users can select from standardized dropdowns during address entry.

Acceptance Criteria

- ☐ `locations` table created in Convex with fields:
 - `_id`
 - `type` (state or lga)
 - `name` (e.g., "Lagos", "Ikeja")
 - `code` (state code only, e.g., "LA", "KD", "FC" for FCT)
 - `parentLocationId` (LGA points to parent state)
 - `stateCode` (for quick filtering)
- ☐ All 36 states + FCT seeded
- ☐ All LGAs seeded (774 total)
- ☐ Indexes: `by_type`, `by_parent`, `by_state`
- ☐ Query: get all states
- ☐ Query: get LGAs by state code

Technical Tasks

- [] Add locations table to convex/schema.ts
- [] Create indexes for efficient queries
- [] Seed script with complete Nigerian states and LGAs
- [] Convex query: locations.getStates()
- [] Convex query: locations.getLgas({ stateCode })
- [] API endpoint: GET /v1/locations/states
- [] API endpoint: GET /v1/locations/states/:stateCode/lgas

Convex Schema

```
// convex/schema.ts

locations: defineTable({
  type: v.union(v.literal("state"), v.literal("lga")),
  name: v.string(),
  code: v.optional(v.string()), // state code: "LA", "KD", "FC"
  parentLocationId: v.optional(v.id("locations")), // LGA references state
  stateCode: v.string(),
})
.index("by_type", ["type"])
.index("by_parent", ["parentLocationId"])
.index("by_state", ["stateCode"]);
```

Seed Data Structure

```
// convex/scripts/seedNigerianLocations.ts

const NIGERIAN_STATES = [
  { name: "Abia", code: "AB", lgas: ["Aba North", "Aba South", "Arochukwu", "Bende",
  { name: "Adamawa", code: "AD", lgas: ["Damsa", "Fufure", "Ganye", "Gayuk", "Gombi"
  // ... all 36 states + FCT
  { name: "FCT", code: "FC", lgas: ["Abaji", "Bwari", "Gwagwalada", "Kuje", "Kwali",
  { name: "Lagos", code: "LA", lgas: ["Agege", "Ajeromi-Ifelodun", "Alimosho", "Amuwa
];

export const seed = mutation({
  args: {},
  handler: async (ctx) => {
    // Insert all states
    for (const state of NIGERIAN_STATES) {
      const stateId = await ctx.db.insert("locations", {
        type: "state",
        name: state.name,
        code: state.code,
        parentLocationId: null,
        stateCode: state.code,
      });
    }
  }
});
```

```

    // Insert all LGAs for this state
    for (const lga of state.lgas) {
      await ctx.db.insert("locations", {
        type: "lga",
        name: lga,
        code: null,
        parentLocationId: stateId,
        stateCode: state.code,
      });
    }
  }
}
});

```

User Story 1.2: Location API & Frontend Components

As a frontend developer, I need location APIs and dropdown components so users can easily select their state and LGA.

Acceptance Criteria

- ☐ API returns all states sorted alphabetically
- ☐ API returns LGAs filtered by selected state
- ☐ Frontend: State dropdown component (all states)
- ☐ Frontend: LGA dropdown component (filtered by state)
- ☐ Loading states while fetching
- ☐ Error handling if API fails
- ☐ Caching: location data cached (rarely changes)

Technical Tasks

- ☐ Convex query: `locations.getStates()`
- ☐ Convex query: `locations.getLgas({ stateCode })`
- ☐ Frontend: `StateSelector` component
- ☐ Frontend: `LgaSelector` component
- ☐ Cache location data in frontend (React Query or similar)

Frontend Components

```

// components/StateSelector.tsx

export function StateSelector({ value, onChange, error }) {
  const states = useQuery(api.locations.getStates);

  if (!states) return <div>Loading states...</div>;
}

```

```

return (
  <div>
    <label>State</label>
    <select
      value={value}
      onChange={(e) => onChange(e.target.value)}
      className="input"
    >
      <option value="">Select State</option>
      {states.map((state) => (
        <option key={state._id} value={state.code}>
          {state.name}
        </option>
      ))}
    </select>
    {error && <p className="text-red-500 text-sm">{error}</p>}
  </div>
);
}

// components/LgaSelector.tsx

export function LgaSelector({ stateCode, value, onChange, error }) {
  const lgas = useQuery(api.locations.getLgas, { stateCode });

  if (!stateCode) {
    return (
      <div>
        <label>LGA</label>
        <select disabled className="input" placeholder="Select state first">
          <option>Select state first</option>
        </select>
      </div>
    );
  }

  if (!lgas) return <div>Loading LGAs...</div>;

  return (
    <div>
      <label>LGA</label>
      <select
        value={value}
        onChange={(e) => onChange(e.target.value)}
        className="input"
        disabled={!stateCode}
      >
        <option value="">Select LGA</option>
        {lgas.map((lga) => (
          <option key={lga._id} value={lga.name}>
            {lga.name}
          </option>
        ))}
      </select>
      {error && <p className="text-red-500 text-sm">{error}</p>}
    </div>
  );
}

```

```
    </div>
  );
}
```

Phase 2: User Phone Verification (Days 3-7)

User Story 2.1: User Phone Number Schema

As a system architect, I need a phone number data model so users can add and verify their phone numbers.

Acceptance Criteria

- [] `userPhones` table created in Convex with fields:
 - `_id`
 - `userId` (FK to users)
 - `phone` (normalized: "234801234567")
 - `phoneNormalized` (for indexing)
 - `isVerified` (boolean)
 - `isPrimary` (boolean, one primary per user)
 - `verificationCode` (hashed OTP)
 - `codeExpiresAt` (timestamp)
 - `verifiedAt` (timestamp)
 - `createdAt` (timestamp)
- [] Indexes: `by_user`, `by_phone`
- [] Validation: Nigerian phone format only
- [] Normalization: convert 08012345678 → 234801234567

Technical Tasks

- [] Add `userPhones` table to `convex/schema.ts`
- [] Create indexes for efficient queries
- [] Phone validation utility function
- [] Phone normalization utility function
- [] Convex query: `userPhones.getByUser({ userId })`
- [] Convex mutation: `userPhones.create`
- [] Convex mutation: `userPhones.verify`
- [] Convex mutation: `userPhones.setPrimary`
- [] Convex mutation: `userPhones.delete`

Convex Schema

```
// convex/schema.ts

userPhones: defineTable({
  userId: v.id("users"),
  phone: v.string(),
  phoneNormalized: v.string(),
  isVerified: v.boolean(),
  isPrimary: v.boolean(),
  verificationCode: v.optional(v.string()), // hashed
  codeExpiresAt: v.optional(v.number()),
  verifiedAt: v.optional(v.number()),
  createdAt: v.number(),
})
.index("by_user", ["userId"])
.index("by_phone", ["phoneNormalized"]);
```

Phone Validation Utility

```
// backend/src/utils/phoneValidation.ts

const NIGERIAN_PREFIXES = [
  "701", "702", "703", "704", "705", "706", "707", "708", "709",
  "801", "802", "803", "804", "805", "806", "807", "808", "809",
  "810", "811", "812", "813", "814", "815", "816", "817", "818", "819",
  "901", "902", "903", "904", "905", "906", "907", "908", "909",
  "912", "913", "915", "916"
];

export function validateNigerianPhone(phone: string): { valid: boolean; error?: string } {
  // Remove all non-digits
  const digits = phone.replace(/\D/g, '');

  // Handle different formats
  let normalized = digits;
  if (digits.startsWith('0') && digits.length === 11) {
    normalized = '234' + digits.slice(1);
  } else if (digits.startsWith('234') && digits.length === 13) {
    normalized = digits;
  } else if (digits.length === 10) {
    normalized = '234' + digits;
  } else {
    return { valid: false, error: 'Phone number must be 10 digits' };
  }

  // Check length (should be 13 digits: 234 + 10)
  if (normalized.length !== 13) {
    return { valid: false, error: 'Invalid phone number length' };
  }

  // Check prefix
  const prefix = normalized.slice(3, 6); // e.g., "801" from "234801234567"
  if (!NIGERIAN_PREFIXES.includes(prefix)) {
```

```

    return { valid: false, error: 'Invalid Nigerian phone prefix' };
  }

  return { valid: true, normalized };
}

export function formatPhoneForDisplay(phone: string): string {
  // 234801234567 → 0801 234 567
  if (phone.startsWith('234')) {
    const withoutCountry = phone.slice(3);
    return `${withoutCountry.slice(0, 3)} ${withoutCountry.slice(3, 6)} ${withoutCountry.slice(6, 9)}`;
  }
  return phone;
}

```

User Story 2.2: User Phone Verification Flow

As a user, I want to verify my phone number during onboarding so I can recover my account and receive important notifications.

Acceptance Criteria

- ☐ Phone number field in profile setup (after email verification)
- ☐ Input format: +234 XXX XXX XXXX (auto-formatted)
- ☐ Real-time validation as user types
- ☐ "Send OTP" button (disabled until valid phone)
- ☐ 6-digit OTP input with auto-focus
- ☐ OTP expires in 10 minutes
- ☐ Countdown timer: "Resend OTP in 59s"
- ☐ Resend OTP button enabled after 60 seconds
- ☐ Max 3 OTP attempts before lockout (15 minutes)
- ☐ Success message: "Phone verified successfully"
- ☐ Verified phone shown in profile with ✓ badge
- ☐ Can change phone (requires re-verification)
- ☐ Can have multiple phones (one primary)

Technical Tasks

- ☐ Frontend: Phone input component with auto-formatting
- ☐ Frontend: OTP verification modal
- ☐ Backend: POST /v1/users/me/phone/send-otp
- ☐ Backend: POST /v1/users/me/phone/verify-otp
- ☐ SMS integration (Termii or Twilio)

- [] Rate limiting: 3 OTPs per hour per phone
- [] Convex mutation: generate and store hashed OTP
- [] Convex mutation: verify OTP and mark as verified
- [] Email notification: "Your phone was verified"

SMS Integration

```
// backend/src/services/smsService.ts

import axios from 'axios';

export async function sendSmsOtp(phone: string, code: string) {
  // Using Termii (Nigerian SMS provider)
  const response = await axios.post(
    'https://api.termii.com/api/sms/send',
    {
      to: phone,
      from: 'Orviohub',
      sms: `Your Orviohub verification code is: ${code}. Valid for 10 minutes.`,
      type: 'plain',
      api_key: process.env.TERMII_API_KEY,
    }
  );

  return response.data;
}

// Alternative: Twilio
export async function sendSmsOtpTwilio(phone: string, code: string) {
  const client = require('twilio')(
    process.env.TWILIO_ACCOUNT_SID,
    process.env.TWILIO_AUTH_TOKEN
  );

  await client.messages.create({
    body: `Your Orviohub verification code is: ${code}. Valid for 10 minutes.`,
    from: process.env.TWILIO_PHONE_NUMBER,
    to: `+${phone}`,
  });
}
```

Backend API

```
// backend/src/routes/users/phone.ts

export async function userPhoneRoutes(fastify: FastifyInstance) {
  // Send OTP
  fastify.post('/v1/users/me/phone/send-otp', {
    preHandler: [authMiddleware],
    handler: async (request, reply) => {
      const { phone } = request.body as { phone: string };
    }
  });
}
```



```

const userId = request.user.id;

// Validate phone
const validation = validateNigerianPhone(phone);
if (!validation.valid) {
  return reply.code(400).send({ error: validation.error });
}

// Check rate limit
const recentOtps = await request.convex.query('userPhones.countRecentOtps', {
  userId,
  phone: validation.normalized,
  minutes: 60
});

if (recentOtps >= 3) {
  return reply.code(429).send({ error: 'Too many OTP requests. Try again in 1 h' });
}

// Generate OTP
const otp = Math.floor(100000 + Math.random() * 900000).toString(); // 6-digit
const otpHash = await bcrypt.hash(otp, 10);
const expiresAt = Date.now() + (10 * 60 * 1000); // 10 minutes

// Store OTP
await request.convex.mutation('userPhones.createOrUpdate', {
  userId,
  phone: validation.normalized,
  verificationCode: otpHash,
  codeExpiresAt: expiresAt,
});

// Send SMS
await sendSmsOtp(validation.normalized, otp);

return reply.send({ success: true, message: 'OTP sent successfully' });
}
});

// Verify OTP
fastify.post('/v1/users/me/phone/verify-otp', {
  preHandler: [authMiddleware],
  handler: async (request, reply) => {
    const { phone, otp } = request.body as { phone: string; otp: string };
    const userId = request.user.id;

    const validation = validateNigerianPhone(phone);
    if (!validation.valid) {
      return reply.code(400).send({ error: validation.error });
    }

    // Get phone record
    const phoneRecord = await request.convex.query('userPhones.getByPhone', {
      userId,
      phone: validation.normalized
    });

```

```

    if (!phoneRecord) {
      return reply.code(400).send({ error: 'Phone not found. Send OTP first.' });
    }

    // Check expiry
    if (phoneRecord.codeExpiresAt && Date.now() > phoneRecord.codeExpiresAt) {
      return reply.code(400).send({ error: 'OTP expired. Request a new one.' });
    }

    // Verify OTP
    const isValid = await bcrypt.compare(otp, phoneRecord.verificationCode);
    if (!isValid) {
      // Increment attempt counter (implement this)
      return reply.code(400).send({ error: 'Invalid OTP' });
    }

    // Mark as verified
    await request.convex.mutation('userPhones.verify', {
      phoneId: phoneRecord._id,
    });

    return reply.send({ success: true, message: 'Phone verified successfully' });
  }
});
}

```

User Story 2.3: User Phone Management

As a user, I want to manage my phone numbers in my profile so I can add backup numbers or change my primary number.

Acceptance Criteria

- [] Profile settings has "Phone Numbers" section
- [] Shows all added phones with verification status
- [] Primary phone marked with badge
- [] "Add Phone" button
- [] Can delete unverified phones anytime
- [] Can delete verified phones (must have at least one verified)
- [] Can set primary phone (must be verified)
- [] Cannot delete last verified phone
- [] Delete confirmation: "Are you sure?"

Technical Tasks

- ☐ Frontend: Phone management UI
- ☐ Backend: GET `/v1/users/me/phones`
- ☐ Backend: DELETE `/v1/users/me/phones/:phoneId`
- ☐ Backend: POST `/v1/users/me/phones/:phoneId/set-primary`
- ☐ Validation: cannot delete last verified phone
- ☐ Audit log: `user.phone_added`, `user.phone_verified`, `user.phone_removed`

Phase 3: Organization Phone & Address (Days 8-12)

User Story 3.1: Organization Phone Schema

As a system architect, I need an organization phone data model so businesses can have verified contact numbers.

Acceptance Criteria

- ☐ `organizationPhones` table created in Convex
- ☐ Fields: `workspaceId`, `phone`, `phoneNormalized`, `isVerified`, `verificationCode`, `codeExpiresAt`, `verifiedAt`, `createdAt`
- ☐ Indexes: `by_workspace`, `by_phone`
- ☐ Can use owner's verified phone (skip verification)
- ☐ Or add new business phone (requires verification)
- ☐ One primary phone per organization

Technical Tasks

- ☐ Add `organizationPhones` table to `convex/schema.ts`
- ☐ Create indexes
- ☐ Convex query: `organizationPhones.getByWorkspace({ workspaceId })`
- ☐ Convex mutation: `organizationPhones.create`
- ☐ Convex mutation: `organizationPhones.verify`
- ☐ API: POST `/v1/workspaces/:workspaceId/phones/send-otp`
- ☐ API: POST `/v1/workspaces/:workspaceId/phones/verify-otp`

Convex Schema

```
// convex/schema.ts

organizationPhones: defineTable({
  workspaceId: v.id("workspaces"),
  phone: v.string(),
  phoneNormalized: v.string(),
  isVerified: v.boolean(),
  verificationCode: v.optional(v.string()),
  codeExpiresAt: v.optional(v.number()),
  verifiedAt: v.optional(v.number()),
  createdAt: v.number(),
})
.index("by_workspace", ["workspaceId"])
.index("by_phone", ["phoneNormalized"]);
```

User Story 3.2: Organization Address Schema

As a system architect, I need an organization address data model so businesses can have structured Nigerian addresses.

Acceptance Criteria

- [] organizationAddresses table created in Convex
- [] Fields:
 - workspaceId
 - type (headquarters, billing, shipping)
 - country (fixed: "Nigeria")
 - state (from dropdown)
 - lga (from dropdown)
 - city (input)
 - area (optional, input)
 - street (input)
 - blockNumber (input)
 - landmark (optional, input)
 - postalCode (optional, input)
 - isDefault (boolean)
 - createdAt, updatedAt
- [] Indexes: by_workspace, by_workspace_type
- [] One default address per organization
- [] Can add multiple addresses (future)

Technical Tasks

- [] Add organizationAddresses table to convex/schema.ts
- [] Create indexes
- [] Convex query: organizationAddresses.getByWorkspace({ workspaceId })
- [] Convex mutation: organizationAddresses.create
- [] Convex mutation: organizationAddresses.update
- [] API: POST /v1/workspaces/:workspaceId/addresses
- [] API: PATCH /v1/workspaces/:workspaceId/addresses/:addressId

Convex Schema

```
// convex/schema.ts

organizationAddresses: defineTable({
  workspaceId: v.id("workspaces"),
  type: v.union(v.literal("headquarters"), v.literal("billing"), v.literal("shipping"),
  country: v.string(), // "Nigeria"
  state: v.string(),
  lga: v.string(),
  city: v.string(),
  area: v.optional(v.string()),
  street: v.string(),
  blockNumber: v.string(),
  landmark: v.optional(v.string()),
  postalCode: v.optional(v.string()),
  isDefault: v.boolean(),
  createdAt: v.number(),
  updatedAt: v.number(),
})
.index("by_workspace", ["workspaceId"])
.index("by_workspace_type", ["workspaceId", "type"]);
```

User Story 3.3: Organization Setup - Phone & Address

As a user creating an organization, I want to add verified business contact details so customers can reach me.

Acceptance Criteria

- [] Organization setup wizard includes "Contact Details" step
- [] Phone section:
 - Option 1: "Use my verified phone: 0801 234 567" (checkbox)
 - Option 2: "Enter new business phone" (input + OTP verification)
- [] Address section:

- Country: "Nigeria" (fixed, not editable)
- State: dropdown (all 36 states + FCT)
- LGA: dropdown (filtered by state)
- City: input
- Area: optional input
- Street: input (required)
- Block Number: input (required)
- Landmark: optional input
- Postal Code: optional input
- [] Validation: all required fields must be filled
- [] Phone must be verified before proceeding
- [] Success: organization created with verified phone and address

Technical Tasks

- [] Frontend: Organization setup wizard step
- [] Phone selection component (use existing or new)
- [] Address form component (State → LGA → City → Street → Block)
- [] OTP verification flow (reuse user phone OTP)
- [] Backend: POST /v1/workspaces/:workspaceId/phones/send-otp
- [] Backend: POST /v1/workspaces/:workspaceId/phones/verify-otp
- [] Backend: POST /v1/workspaces/:workspaceId/addresses

Frontend Component

```
// surfaces/launcher/pages/OrganizationSetup/ContactDetailsStep.tsx

export function ContactDetailsStep({ onComplete }) {
  const { user } = useAuthStore();
  const [useMyPhone, setUseMyPhone] = useState(true);
  const [businessPhone, setBusinessPhone] = useState('');
  const [phoneVerified, setPhoneVerified] = useState(false);

  const [address, setAddress] = useState({
    country: 'Nigeria',
    state: '',
    lga: '',
    city: '',
    area: '',
    street: '',
    blockNumber: '',
    landmark: '',
    postalCode: ''
  });
}
```

```

const handleContinue = async () => {
  // Validate phone
  const finalPhone = useMyPhone ? user.primaryPhone : businessPhone;
  if (!finalPhone || !phoneVerified) {
    alert('Please verify your business phone');
    return;
  }

  // Validate address
  if (!address.state || !address.lga || !address.street || !address.blockNumber) {
    alert('Please fill all required address fields');
    return;
  }

  // Save to organization
  await saveOrganizationContact({
    phone: finalPhone,
    isPhoneVerified: useMyPhone || phoneVerified,
    address
  });

  onComplete();
};

return (
  <div className="space-y-6">
    <div>
      <h2 className="text-xl font-semibold mb-4">Business Contact Details</h2>

      {/* Phone Section */}
      <div className="card p-4 mb-4">
        <h3 className="font-medium mb-3">Business Phone Number</h3>

        <label className="flex items-center space-x-2 mb-3">
          <input
            type="checkbox"
            checked={useMyPhone}
            onChange={(e) => setUseMyPhone(e.target.checked)}
          />
          <span>Use my verified phone: {formatPhoneForDisplay(user.primaryPhone)}</span>
        </label>

        {!useMyPhone && (
          <div className="ml-6">
            <PhoneInput
              value={businessPhone}
              onChange={setBusinessPhone}
              onVerified={() => setPhoneVerified(true)}
            />
          </div>
        )}
      </div>

      {/* Address Section */}
      <div className="card p-4">

```

```

        <h3 className="font-medium mb-3">Business Address</h3>

        <AddressForm
          value={address}
          onChange={setAddress}
          countryFixed={true}
        />
      </div>
    </div>

    <Button onClick={handleContinue} disabled={!phoneVerified}>
      Continue
    </Button>
  </div>
);
}

```

User Story 3.4: Organization Contact Management

As an organization owner, I want to update my business phone and address so I can keep contact information current.

Acceptance Criteria

- [] Organization settings has "Contact Details" section
- [] Shows current verified phone
- [] "Change Phone" button (requires re-verification)
- [] Shows current address
- [] "Edit Address" button
- [] Can add additional addresses (billing, shipping)
- [] Can set default address
- [] All changes audited

Technical Tasks

- [] Frontend: Organization contact settings page
- [] Backend: PATCH /v1/workspaces/:workspaceId/phones/:phoneId
- [] Backend: PATCH /v1/workspaces/:workspaceId/addresses/:addressId
- [] Backend: POST /v1/workspaces/:workspaceId/addresses (add new)
- [] Audit log: organization.phone_changed, organization.address_updated

Phase 4: Branch Phone & Address (Days 13-14)

User Story 4.1: Branch Contact Schema

As a system architect, I need branch-level phone and address fields so each location can have its own contact details.

Acceptance Criteria

- [] Extend branches table with contact fields
- [] Phone: optional, verified if provided
- [] Address: same structure as organization address
- [] Default: copy from organization (editable)
- [] Can be different from org address

Technical Tasks

- [] Update branches table schema
- [] Add fields: phone, phoneNormalized, isPhoneVerified, country, state, lga, city, area, street, blockNumber, landmark
- [] Convex mutation: branches.updateContact
- [] API: PATCH /v1/workspaces/:workspaceId/branches/:branchId/contact

Convex Schema Update

```
// convex/schema.ts

branches: defineTable({
  workspaceId: v.id("workspaces"),
  name: v.string(),
  code: v.string(),
  isPrimary: v.boolean(),
  status: v.union(v.literal("active"), v.literal("archived")),

  // Contact fields
  phone: v.optional(v.string()),
  phoneNormalized: v.optional(v.string()),
  isPhoneVerified: v.boolean(),

  // Address fields
  country: v.string(), // "Nigeria"
  state: v.string(),
  lga: v.string(),
  city: v.string(),
  area: v.optional(v.string()),
  street: v.string(),
  blockNumber: v.string(),
  landmark: v.optional(v.string()),
```

```
    createdAt: v.number(),
    updatedAt: v.number(),
  })
  .index("by_workspace", ["workspaceId"]);
```

User Story 4.2: Branch Contact Setup

As a user creating a branch, I want to add branch-specific contact details so customers can reach this location.

Acceptance Criteria

- ☐ Branch creation form includes optional phone & address
- ☐ Phone: checkbox "This branch has a phone number"
- ☐ If checked: can use org phone OR enter new branch phone
- ☐ If new phone: verify with OTP
- ☐ Address: pre-fill from organization (editable)
- ☐ All fields optional except branch name
- ☐ Success: branch created with contact details

Technical Tasks

- ☐ Frontend: Branch creation form with contact section
- ☐ Backend: POST /v1/workspaces/:workspaceId/branches (include contact fields)
- ☐ OTP verification for new branch phone

Testing Checklist

Unit Tests

- ☐ Phone validation works for all Nigerian formats
- ☐ Phone normalization correct (0801... → 234801...)
- ☐ OTP generation and verification works
- ☐ Rate limiting prevents OTP abuse
- ☐ Address validation requires all mandatory fields
- ☐ State/LGA dropdowns populated correctly

Integration Tests

- ☐ User can verify phone during onboarding
- ☐ Organization can use owner's verified phone
- ☐ Organization can verify new business phone
- ☐ Address saved with correct state/LGA structure
- ☐ Branch inherits org address by default
- ☐ Branch can have different address from org

E2E Tests

- ☐ Complete onboarding with phone verification
- ☐ Create organization with verified phone and address
- ☐ Create branch with custom contact details
- ☐ Update organization contact details
- ☐ Change branch phone (re-verify)

Deployment Plan

Days 1-2: Location Database

- ☐ Deploy locations table
- ☐ Seed Nigerian states and LGAs
- ☐ Deploy location APIs
- ☐ Test: state/LGA dropdowns work

Days 3-7: User Phone Verification

- ☐ Deploy userPhones table
- ☐ Deploy OTP send/verify APIs
- ☐ Integrate SMS provider (Termii)
- ☐ Deploy phone verification UI
- ☐ Test: user can verify phone

Days 8-12: Organization Contact

- ☐ Deploy organizationPhones and organizationAddresses tables
- ☐ Deploy org phone/address APIs
- ☐ Update organization setup wizard
- ☐ Deploy org contact settings

- ☐ Test: org can add verified phone and address

Days 13-14: Branch Contact

- ☐ Update branches table with contact fields
- ☐ Deploy branch contact APIs
- ☐ Update branch creation form
- ☐ Test: branch can have custom contact

Success Metrics

- ✓ 90%+ of users verify phone during onboarding
- ✓ OTP delivery rate > 95%
- ✓ Address completion rate > 80%
- ✓ State/LGA dropdowns load in < 500ms
- ✓ Phone verification completes in < 2 minutes
- ✓ No PII (personal identifiable information) leaks

Security & Compliance

Phone Number Security

- ☐ Phone numbers encrypted at rest
- ☐ OTP codes hashed (bcrypt)
- ☐ Rate limiting: 3 OTPs per hour
- ☐ Audit logs: all phone changes logged
- ☐ No phone numbers in client-side logs

Address Privacy

- ☐ Addresses visible only to org members
- ☐ Public addresses (on receipts) show only necessary fields
- ☐ No geocoding without consent
- ☐ GDPR/NDPR compliance (right to deletion)

Out of Scope (For Now)

- **✗** International phone numbers (Nigeria only)
- **✗** International addresses (Nigeria only)
- **✗** Multiple addresses per org (one for now)
- **✗** Address verification APIs (unreliable in Nigeria)
- **✗** Geocoding (lat/lng coordinates)
- **✗** WhatsApp OTP (SMS only for MVP)
- **✗** Voice call OTP (fallback later)

This implementation plan is ready for development. All user stories must be completed for phone verification and address management to be considered functional.